# Table of contents



# Chapter 9. Wiring your iPhone into your Linux machines

> **The payoff of this chapter:** move files between your iPhone and your Linux laptop in one tap, share a clipboard and see your phone's battery on the desktop, do the same to your Pi from anywhere over Tailscale, and know the one wired fallback (USB) for when Wi-Fi isn't an option, without installing a pile of tools you'll never use.

You've built a homelab your iPhone can reach. This part closes the loop in the *other* direction: getting your phone and your Linux machines to talk to each other for the everyday stuff, "send me that PDF," "paste this URL onto my desktop," "pull these photos off my phone." On a Mac you'd reach for AirDrop and Handoff; on Linux you assemble the same conveniences from two small apps, plus one wired option you'll rarely need.

The important framing up front: **these are three different tools solving three different problems.** Don't think "which one wins", think "which job am I doing."

## The two Wi-Fi tools, and why you want both

| Job | LocalSend | KDE Connect (on iPhone) |
|---|---|---|
| Send files phone → Linux | | |
| Receive files Linux → phone | | |
| Share clipboard | | |
| Phone battery on desktop | | |
| Media remote control | | |
| **Mirror phone notifications to desktop** | | **on iPhone** (Android-only) |

| Job | LocalSend | KDE Connect (on iPhone) |
|---|---|---|
| SMS / run commands from desktop<br>Dead-simple, one-purpose app | | **on iPhone** (Android-only) |

**!** The iPhone reality check (this is where most guides mislead you)

KDE Connect's headline features, notification mirroring, replying to texts from your desktop, running commands on the phone, are **Android-only**. Apple's sandbox forbids a third-party app from reading other apps' notifications or sending SMS, so on iOS none of that works no matter what a generic comparison table says. What the **iOS** KDE Connect app (v0.5.x as of 2026) *does* deliver is real and useful: file transfer, **clipboard sync**, **battery status**, and media remote control. One more iOS quirk: Apple's background limits mean the KDE Connect app must be **open or recently used** to stay reachable. It can't sit dormant for days and still answer.

### LocalSend: your AirDrop replacement

Think of LocalSend as a dedicated screwdriver: it does exactly one thing, extremely well. Open it, pick a nearby device, send the file. No pairing, no account, no integration, and it cheerfully moves a 5 GB video that you'd never want to push through a more "integrated" tool.

That's its entire job, and it's the right tool whenever the task is purely "get this file from here to there."

### KDE Connect: your phone companion

KDE Connect is the multitool. File transfer is just one of its features; the reason to install it is the *integration*, on iPhone specifically, that means a **shared clipboard** (copy `192.168.1.50` on your phone, paste it on your desktop) and **battery status** on your desktop panel. The notification/SMS superpowers are Android's; on iOS you're here for clipboard and file flow.

### "If KDE Connect transfers files, why install LocalSend?"

You don't *have* to, plenty of people run only KDE Connect. The reason many keep LocalSend around anyway is sheer simplicity: when you just want to fling a big file across the room, LocalSend is two taps with zero ceremony. It's like keeping a dedicated screwdriver even though your multitool has one. Disk is cheap; install both and reach for whichever fits the moment.

## Install them

### On the iPhone

Both are free on the App Store: search **LocalSend** and **KDE Connect** (the KDE Connect one is published by KDE e.V.). Install both.

### On your Linux laptop

**LocalSend**, easiest via Flatpak (works the same on Arch and Ubuntu):

```
flatpak install flathub org.localsend.localsend_app
```

(Or grab the AppImage from the project's GitHub releases if you don't use Flatpak.)

**KDE Connect**, from your distro's repos:

```
# Arch
sudo pacman -S kdeconnect

# Ubuntu/Debian
sudo apt install kdeconnect
```

> **Running KDE Connect on Hyprland (no Plasma, no GNOME)**
>
> KDE Connect does **not** require the full KDE Plasma desktop, but it does need its background daemon running and a way to interact with it. On a bare window manager like Hyprland:
>
> - The daemon `kdeconnectd` is started on demand over D-Bus; launching the GUI (`kdeconnect-app`) or running `kdeconnect-cli -l` brings it up.
> - For an at-a-glance tray/battery indicator, add a **Waybar** "custom" module that calls `kdeconnect-cli`, or just keep `kdeconnect-app` a keybind away.
> - Skip **GSConnect**, it's a GNOME Shell extension and won't help you on Hyprland. Plain `kdeconnect` is the right choice here.

### Open the firewall

Both tools need their ports reachable on the LAN. If you run `ufw`:

```
sudo ufw allow 53317/udp        # LocalSend discovery + transfer
sudo ufw allow 53317/tcp
sudo ufw allow 1714:1764/udp    # KDE Connect
sudo ufw allow 1714:1764/tcp
```

If a transfer "sees" the device but hangs at 0%, a closed firewall port is the first thing to check.

#### Pair

- **LocalSend:** no pairing, open it on both devices, on the same Wi-Fi, and they appear in each other's list. Send.
- **KDE Connect:** open the app on both, tap the laptop in the phone's device list, and **accept the pairing request** on the desktop (`kdeconnect-app` or the CLI will prompt). Pairing is a one-time trust handshake.

### Making it work *anywhere*: over Tailscale

Here's the homelab tie-in. Out of the box, both tools find devices using **LAN multicast/broadcast discovery**, great at home, useless the moment your phone is on cellular, because broadcast packets don't traverse a VPN. But your tailnet gives every device a stable `100.x.y.z` address, and both apps let you **add a device by IP manually**, which sidesteps discovery entirely:

- **LocalSend:** Settings → **add a favorite / manual device** → enter the target's Tailscale IP (e.g. your laptop at `100.x.y.z`) with port `53317`. Now you can send to your laptop from the airport.
- **KDE Connect:** the iOS app has an **"Add device by IP"** / refresh option, enter the laptop's Tailscale IP. Pair once, and clipboard + file transfer work across the tailnet.

> Why this is the natural payoff of everything before it
>
> This is the same trick that powered the whole series: discovery and "same network" assumptions break the instant you leave home, and Tailscale's stable addresses are the fix. Just as you reached Audiobookshelf and `https://home.home` by tailnet IP, you reach your *laptop* by tailnet IP here. The phone Linux conveniences stop being a home-only luxury.

### Phone ↔ the headless Pi

A caveat worth stating plainly: LocalSend and KDE Connect are **desktop** tools. Your Pi from Chapters 1–5 runs a **headless** server OS (Ubuntu Server 26.04 LTS), no graphical desktop, so neither app is the natural fit there. For moving a file between your phone and the Pi, you already have better, GUI-free paths:

- **Drop it into a service you already run.** Files you want *on* the Pi (more audiobooks, say) can go straight into `~/audiobookshelf/media/Audiobooks`, `scp` them over the tailnet:

  ```
  # from your laptop, over Tailscale, no LAN required
  scp ~/Downloads/lesson.mp3
  you@homelab:~/audiobookshelf/media/Audiobooks/
  ```

- **From the phone directly**, an iOS file-manager app that speaks SSH/SFTP (several exist) can connect to `homelab` over Tailscale and drop files onto the Pi without any desktop app in between.

In short: **laptop ↔ phone** is LocalSend/KDE Connect territory; **phone/laptop ↔ the headless Pi** stays SSH/`scp` territory, which you already have for free over Tailscale.

### The wired fallback: USB access with libimobiledevice

Everything above is Wi-Fi. USB is a **completely separate technology stack**, and (honestly) most people rarely need it. Install it the day you actually hit one of these, not before:

- No Wi-Fi available and you need photos off the phone *now*.
- You're moving 100 GB of video and want the fastest possible transfer.
- The phone's networking is broken and you need emergency access.

When that day comes:

```
# Arch
sudo pacman -S libimobiledevice ifuse usbmuxd

# Ubuntu/Debian
sudo apt install libimobiledevice6 libimobiledevice-utils ifuse usbmuxd
```

Then plug in, trust the computer on the phone, and mount:

```
idevicepair pair                # tap "Trust" on the iPhone when prompted
mkdir -p ~/iphone
ifuse ~/iphone                  # mounts the phone's media (camera roll)
# ... copy your files ...
fusermount -u ~/iphone          # unmount when done
```

> **i** What USB actually gives you (and what it doesn't)
>
> `usbmuxd` is the daemon that talks to the phone over the USB cable; `libimobiledevice` is the library implementing Apple's protocols; `ifuse` mounts the result as a folder. Because of iOS's sandbox, `ifuse` exposes the **photos/ camera-roll area** (and, with extra flags, individual apps' document folders), *not* the whole iOS filesystem. So USB is excellent for bulk photo/video offload, but it isn't a "browse my entire iPhone like a USB stick" experience. No jailbreak, no full filesystem, that's an Apple limitation, not a missing package.

### If I were setting up your machine

Given your stack (iPhone, Hyprland, Arch/Ubuntu, plus the Pi homelab) start with exactly two things:

1. **LocalSend** (the AirDrop replacement)
2. **KDE Connect** (clipboard + battery + files)

…and nothing else. Add the Tailscale manual-IP pairing once you want these to work away from home. Then, *if* six months from now you find yourself saying "I wish I could pull my whole camera roll off over a cable," install `libimobiledevice` + `ifuse`, and at that point you'll understand exactly why you need them, instead of carrying tools you never touch.

For most people in 2026, Wi-Fi transfer (LocalSend / KDE Connect, extended over Tailscale) handles the overwhelming majority of phone  Linux moments. USB is a backup and a power-user tool, not a daily requirement.

## Recap: and the series, complete

- **LocalSend** = AirDrop replacement: dead-simple file send, install on both the iPhone and the laptop.
- **KDE Connect** = phone companion: clipboard sync and battery on iOS (the notification/SMS features are Android-only, don't expect them on iPhone).
- **Tailscale extends both off-LAN:** add devices by their `100.x.y.z` IP so transfer and clipboard work from anywhere, exactly as the rest of your homelab does.
- **The headless Pi** stays on SSH/`scp` over Tailscale, the right tool for a machine with no desktop.
- **USB (`libimobiledevice` + `ifuse`)** is the wired fallback for bulk photo offload or no-Wi-Fi emergencies, install it only when you actually need it.

That completes *Beginner Homelab on a Raspberry Pi*. You started by building a foundation (a hardened, always-on Pi on a private Tailscale network) and then gave it job after job: streaming your media, blocking ads on every device, clean `http://*.home` URLs behind a reverse proxy, a single `https://home.home` control panel, a deliberate answer for VPN privacy on the road, and now two-way file sharing with your phone. Every piece runs on hardware you own, over one private network, with nothing exposed to the public internet.